

P1808R0

Contra P0339 "`polymorphic\_allocator<>` as a vocabulary type"

Published Proposal, 2019-07-08

Author:

[Arthur O'Dwyer](#)

Audience:

LEWG, LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Source:

github.com/Quuxplusone/draft/blob/gh-pages/d1808-contr-p0339.bs

Current:

rawgit.com/Quuxplusone/draft/gh-pages/d1808-contr-p0339.html

Abstract

[P0339R6] "`polymorphic\_allocator<>` as a vocabulary type" (February 2019) went through LWG wording review in Kona this February and has been merged into the C++2a Working Draft [N4810]. It should be reverted; or, if not reverted, we propose some improvements to its usability and safety.

Table of Contents

- 1 P0339's example code
- 2 The `new_object<T>` API
- 3 The interaction with CTAD and common typos

4 Summary of objections to P0339

5 What might a convenience interface look like?

6 If P0339 is not reverted, at least remove the defaulted parameter

7 If P0339 is not reverted, default the first template parameters of `allocate_object` and `new_object`

Appendix A: Proposed straw polls

References

Normative References

Informative References

(For WG21 members, LWG's notes on P0339R6 are [here](#).)

This paper condenses arguments from [\[Contra\]](#).

§ 1. P0339's example code

Okay, so, what does [\[P0339R6\]](#) propose to allow us to write? From the paper: [Here](#) is the "before" code, and [here](#) is the "after" code. P0339 shows a dramatic difference between the "before" version of allocating a linked-list node:

```
using node_alloc = typename alloc_traits::template rebind_alloc<node>;
node_alloc m_alloc = ...;

using alloc_node_traits = typename alloc_traits::template rebind_traits<node>;
node *n = alloc_node_traits::allocate(m_alloc, 1);
alloc_node_traits::construct(m_alloc, &n->m_value, v);
n->m_next = m_head;
```

and the "after" version:

```
using allocator_type = std::pmr::polymorphic_allocator<>;
allocator_type m_alloc = ...;

node *n = m_alloc.allocate_object<node>();
m_alloc.construct(&n->m_value, v);
n->m_next = m_head;
```

However, notice that the "before" version was an STL-style class template taking an allocator parameter, whereas the "after" version is a concrete class type restricted to dealing with only a single allocator type — `std::pmr::polymorphic_allocator`.

So the main difference is that `StringList2` removes the allocator template parameter. Removing template parameters does indeed dramatically simplify code, but you don't need to modify `polymorphic_allocator` to get that benefit! Let's compare how the non-parameterized `StringList2` would look in pure vanilla C++17: [here](#).

```
using node_alloc = std::pmr::polymorphic_allocator<node>;
node_alloc m_alloc = ...;

node *n = m_alloc.allocate(1);
m_alloc.construct(&n->m_value, v);
n->m_next = m_head;
```

The vanilla C++17 version of `StringList2` is actually *simpler* than P0339's proposed C++2a version!

Notice that because `StringList2` is not aware of allocator types other than its hard-coded one, we don't have to go through `allocator_traits` to get at `construct`. We know that `polymorphic_allocator<node>` provides `allocate` and `construct` methods that fit our needs exactly.

§ 2. The `new_object<T>` API

P0339 adds the following member function to all specializations of `polymorphic_allocator<Tp>`:

```
template<class T, class... CtorArgs> T* new_object(CtorArgs&&... ctor_a
```

It is unfortunate that this new API is being proposed only for `std::pmr::polymorphic_allocator`, and not for other concrete allocator types such as `std::allocator` at the same time. Programmers of properly C++11-allocator-aware containers will not be able to take advantage of the `new_object` API at all.

P0339's motivating example can't use the `new_object` API, because `struct node` is not allocator-aware. `struct node` deliberately lacks the constructors that would be needed to pipe the allocator from `m_alloc` down into `node::m_value`. That's why P0339's example code explicitly calls `construct` on the `n->m_value` object, instead of letting it be recursively constructed by `node`'s constructor or by `new_object`.

Earlier revisions of P0339 proposed to add the `new_object` API only to `polymorphic_allocator<std::byte>`. Therefore it needed the caller to supply template parameter `T` in every case:

```
std::polymorphic_allocator<node2> m_alloc = ...;
m_head = m_alloc.new_object<node2>(v, m_head);
```

But since `polymorphic_allocator<Tp>` is already associated with a fixed `Tp`, it would be more concise to write simply

```
std::polymorphic_allocator<node2> m_alloc = ...;
m_head = m_alloc.new_object(v, m_head);
```

This is the motivation for one of our proposed changes to the P0339 `new_object` and `allocate_object` interfaces: that they should default their template parameter `T` to the allocator's value type `Tp`.

§ 3. The interaction with CTAD and common typos

Earlier revisions of P0339 proposed to add the `new_object` API only to `polymorphic_allocator<std::byte>`. Therefore it needed a "convenient" alias for the specialization `polymorphic_allocator<std::byte>` (as opposed to other specializations, which would not have had the new API).

The merged version of P0339 attaches the new API to all specializations of `polymorphic_allocator<Tp>`, yet still proposes to slightly shorten the name of

`polymorphic_allocator<std::byte>` by giving `polymorphic_allocator` a defaulted template parameter.

Before P0339, the following code snippet would be a syntax error. (You forgot the `<T>!`)

```
template<class T>
void *allocate_space_for_n_Ts_with_the_default_resource(int n) {
    std::pmr::polymorphic_allocator alloc;
    return alloc.allocate(n);
}
```

After P0339, thanks to the defaulted template parameter, and partly [thanks to CTAD](#), that code snippet compiles quietly and allocates `n` bytes of memory, rather than the intended `n*sizeof(T)` bytes.

Even in a world without CTAD, accidentally writing `polymorphic_allocator<>` instead of `polymorphic_allocator<T>` is not unthinkable. I have personally observed [Reddit commenters](#) writing `polymorphic_allocator<>` and `memory_resource<>` instead of `polymorphic_allocator<T>` and `memory_resource`.

A significant number of C++ developers are already confused about which of `polymorphic_allocator` and `memory_resource` are templates, which are type-erased, and which are classically polymorphic. Allowing these developers to write `std::pmr::polymorphic_allocator a;` as if it were a concrete class type does them a grave disservice.

Consider the difference between

```
std::pmr::polymorphic_allocator a1 = std::pmr::new_delete_resource();
std::pmr::polymorphic_allocator a2 = std::pmr::vector<int>().get_allocator();
```

Above, `a1::value_type` is `std::byte` but `a2::value_type` is `int`.

§ 4. Summary of objections to P0339

- P0339's convenience functionality is not as optimally designed as it could be.
- P0339's own example shows the inferiority of P0339's `allocate_object<T>` functionality, compared to what's already in C++17.
- P0339 pointlessly privileges `std::byte` over all other `T`.

- P0339 added a default template parameter that interacts badly with CTAD, and serves merely to hide bugs.

§ 5. What might a convenience interface look like?

To get the benefits of P0339's "convenience interface" without the downsides, one might introduce a non-templated `std::pmr::handle` which can be used without messing with the allocator model at all. Using the "handle" model instead of the "allocator" model, we could write a `StringList3` that looks [like this](#):

```
handle m_res = ...;
node *n = m_res.allocate<node>(1);
m_res.construct(&n->m_value, v);
n->m_next = m_head;
```

Here, `m_res` is a data member of type `std::pmr::handle`. It doesn't pretend to be an Allocator, because it doesn't need to. All accesses to its underlying resource go through the new convenience API, never through the C++11 allocator API. The one place where the old allocator API is needed, `StringList3::get_allocator()`, simply returns `StringList3::allocator_type{m_res.resource()}.`

This idea can be implemented in vanilla C++17, entirely in user code. No changes to `std::pmr::polymorphic_allocator` are needed. For this reason, my first preference is that P0339's changes to `std::pmr::polymorphic_allocator` should be completely reverted.

§ 6. If P0339 is not reverted, at least remove the defaulted parameter

Modify [\[mem.poly.allocator.class\]](#) as follows:

```
namespace std::pmr {
    template<class Tp = byte> class polymorphic_allocator {
        memory_resource* memory_rsrc; // exposition only
```

This stops `polymorphic_allocator` from being usable without angle brackets.

Further proposals might then be entertained to introduce a "convenience alias" for `std::pmr::polymorphic_allocator<std::byte>`, or

`std::pmr::polymorphic_allocator<char>`, or `std::pmr::polymorphic_allocator<int>`, or any other "representative" specialization of `polymorphic_allocator`. However, since all specializations of `polymorphic_allocator` have access to P0339's new API, and all specializations of `polymorphic_allocator` are implicitly interconvertible, there is no reason to privilege any one specialization above the others. Furthermore, `std::pmr::polymorphic_allocator<>` is a particularly cumbersome spelling of the "convenience" alias; it could be a proper (non-template) alias such as `using std::pmr::handle = std::pmr::polymorphic_allocator<std::byte>` instead.

§ 7. If P0339 is not reverted, default the first template parameters of `allocate_object` and `new_object`

Modify [\[mem.poly.allocator.class\]](#) as follows:

```
void* allocate_bytes(size_t nbytes, size_t alignment = alignof(max_align_t));
void deallocate_bytes(void* p, size_t nbytes, size_t alignment = alignof(max_align_t));
template<class T == Tp> T* allocate_object(size_t n = 1);
template<class T> void deallocate_object(T* p, size_t n = 1);
template<class T == Tp, class... CtorArgs> T* new_object(CtorArgs&&... ctor_args);
template<class T> void delete_object(T* p);

template<class T, class... Args>
    void construct(T* p, Args&&... args);

template<class T>
    void destroy(T* p);
```

This allows `polymorphic_allocator<Tp>::new_object(args...)` to be used without angle brackets.

Because the value type of the allocator depends on its template argument, this change should not be taken unless the default template argument is removed from `polymorphic_allocator`. If this change were taken without that one, then the following line of code would accidentally construct a `std::byte` instead of the desired `int`:

```
auto *p = std::polymorphic_allocator{}.construct(42);
```

§ Appendix A: Proposed straw polls

	<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
<i>Revert P0339; send these concerns back to the author of P0339.</i>	—	—	—	—	—
<i>Apply the proposed change to remove the default template argument from <code>polymorphic_allocator</code>.</i>	—	—	—	—	—
<i>Apply the proposed changes to remove the default template argument from <code>polymorphic_allocator</code> and to add a default template argument to <code>allocate_object</code> and <code>new_object</code>.</i>	—	—	—	—	—

§ References

§ Normative References

[N4810]

Richard Smith. [Working Draft, Standard for Programming Language C++](https://wg21.link/n4810). 15 March 2019. URL: <https://wg21.link/n4810>

§ Informative References

[Contra]

Arthur O'Dwyer. [Contra P0339 “`polymorphic\_allocator<>` as a vocabulary type”](https://quuxplusone.github.io/blog/2019/07/02/contra-p0339/). July 2019. URL: <https://quuxplusone.github.io/blog/2019/07/02/contra-p0339/>

[P0339R6]

Pablo Halpern; Dietmar Kühl. [polymorphic_allocator<> as a vocabulary type](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0339r6.pdf). February 2019. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0339r6.pdf>